Python for data science

Nicolas Vannson, PhD

TBS - 2026

A bit of presentation?

Nicolas Vannson, PhD

Audiologist @ Brussels in 2010
Master in healthcare @ Montpellier in 2012
PhD in auditory neuroscience @ Toulouse in 2016

Yesterday :
Audiologist / CNRS associate researcher/ lecturer

Today :
Hearing Data Scientist – founder of Hearing Data Science

Courses menu and evaluation?

Courses :
Python for data science
Advanced python for data science (December)

Evaluations :
Python for data science - personal project
Advanced python for data science – group project

Overall :
Questions (MCQ) for your final year project

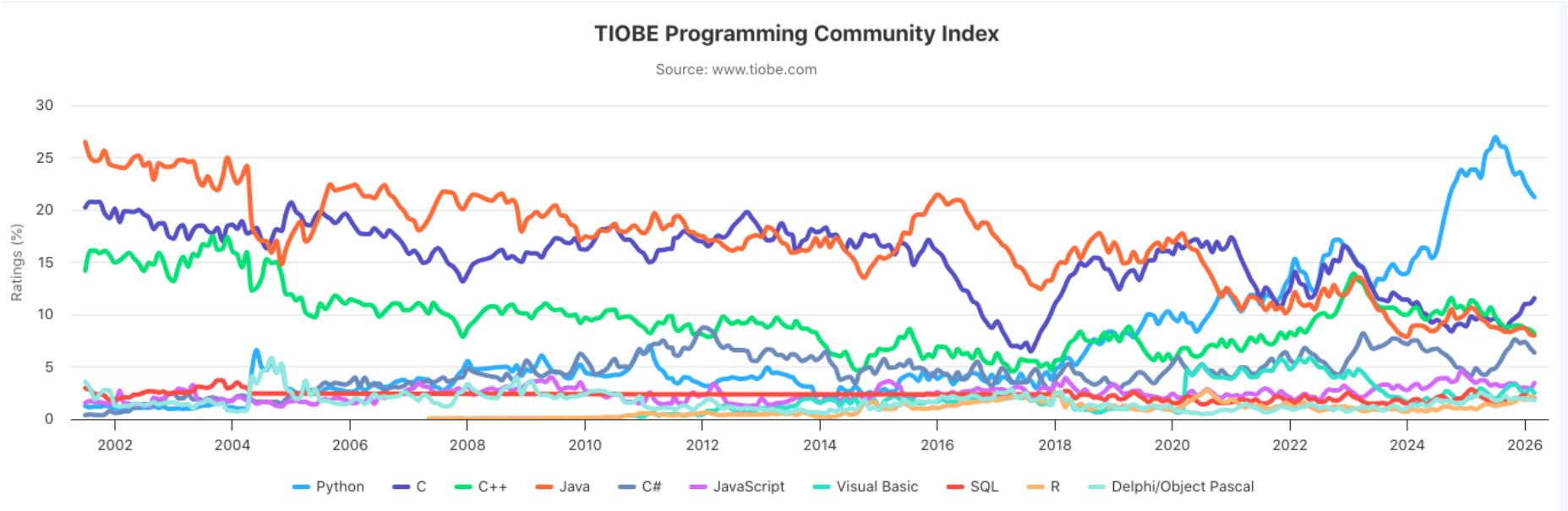
Objectives?

1. Master the basics of data analysis
2. Create data visualisation & manipulating DataFrame
3. Write your own function

WHY PYTHON?



Why Python?


TIOBE Programming Community Index
Source: www.tiobe.com
Ratings (%)
30
25
20
15
10
5
0
2002
2004
2006
2008
2010
2012
2014
2016
2018
2020
2022
2024
2026
Python
C
C++
Java
C#
JavaScript
Visual Basic
SQL
R
Delphi/Object Pascal

Python vs R?

	Python	R
Origin and purpose	General-purpose programming language	Language and environment for statistical computing
Data Analysis	Powerful with Pandas & NumPy.	Data frames built into the language, tidyverse
Data Visualization	Matplotlib, Seaborn, Plotly	ggplot2 (grammar of graphics)
Modelling and deployment	Widely used for machine learning and deep learning (Scikit-Learn, PyTorch, TensorFlow).	Strong in classical statistical modelling. Shiny and plumber for apps and APIs.

“90 %”

Guillaume Saint-Cirgue, Machine Learnia founder

WHY DATA SCIENCE?

Data science a whole process?

Customer needs

Report /dashboard



Pipeline to collect data

ML/DL

https://www.clicdata.com

BASIC PROGRAMMING

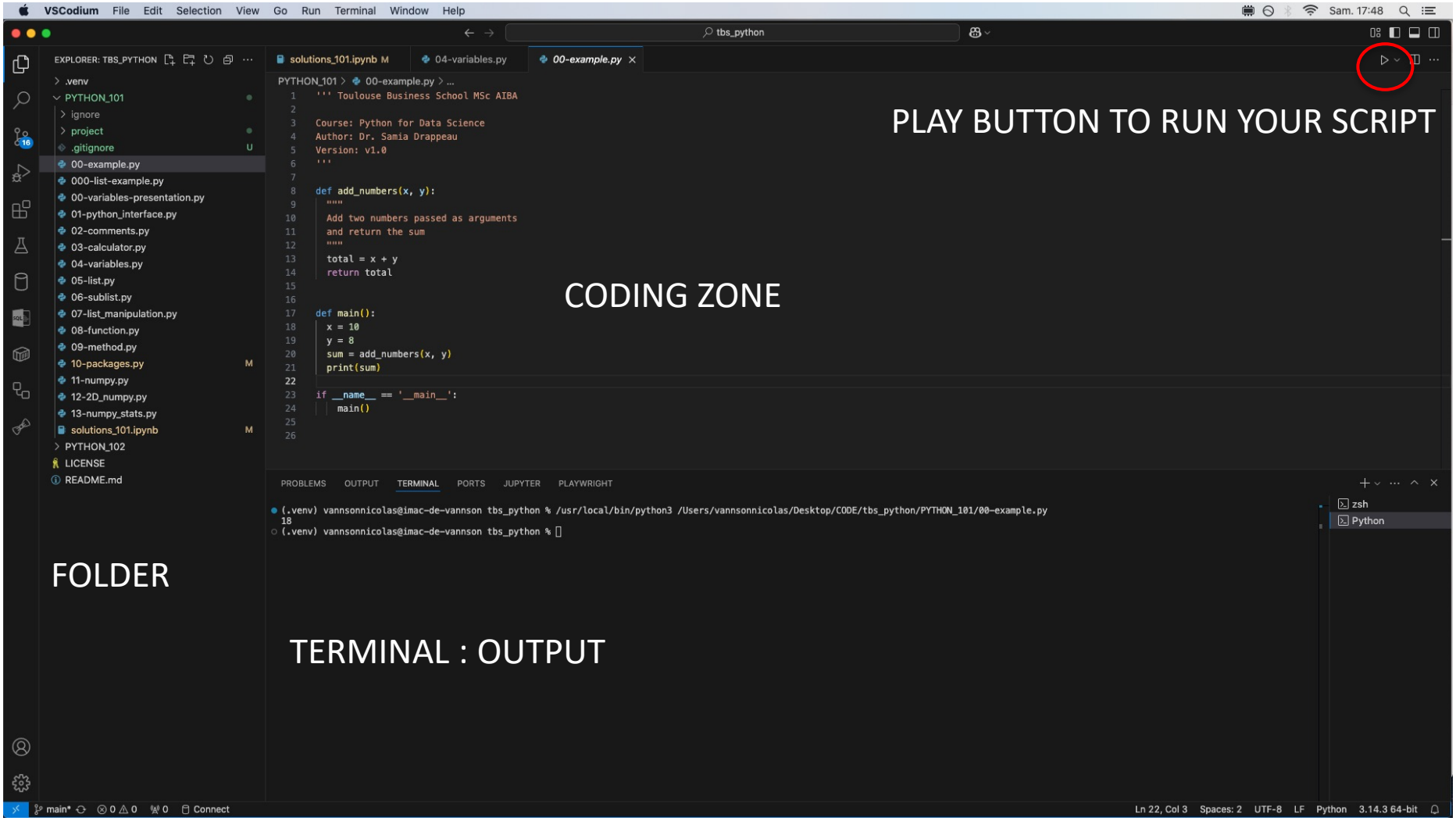


- This course is a hands-on course
- 3 exercises and correction
- 15-20 minutes for all exercises
- Go as your own pace !

LET'S PRACTICE



VS Code the IDE



A python script!

- A python script ends with .py (extension) => terminal is another window
- A python notebook ends with .ipynb (advanced python course)
  terminal below code block

01-python_interface.py

```
# This file is a Python script
# Experiment with it to get familiar with the Python Interface

msg = "Hello, TBS!"
print(msg)

compute = 5/208
print(compute)
```

[1]

```
Hello, TBS!
0.02403846153846154
```

Clone the GitHub repo?

Cloning a Git repository:

- Copy the repository URL: https://github.com/MEMAUDATA/tbs_python.git
- In VS Code Welcome window, click on “Clone Git Repository”.
- Paste the URL into the input field at the top and select Desktop as the destination folder.
- Click on Open (or Open Repository) when prompted.
- In the File Explorer, navigate to the PYTHON_101 folder to access the exercises.

- 01-python_interface.py
- 02-comments.py
- 03-calculator.py

LET'S PRACTICE



Variables and types?

- A variable is a named storage for a value that can change throughout the code

```
a = 2
print(a**2)
>>> 4
```

- Type
  - Numbers : float (3.14) or int (3)
  - Text : str (“2” or ‘2’)
  - Boolean : bool (True or False)

- Different types cause different behaviour

```
2    + 2     >>> 4 (=> int)
"2"  + "2"   >>> "22" (=> str)
```

- Always check the variable’s type to avoid simple mistakes !

```
price = 100
type(price)
>>> int
```

List?

- A list is a named collection of values or variables
- Can contain any type (even a list)
- Example :

```
savings = [100,200,300,400,500]
type(savings)
>>> list
client_savings = ["c1", 100, "c2", 200]
type(client_savings)
>>> list
```

Get access to an item of the list?

- Use **index** to get access to an element of the list. First element starts at 0 (not 1) and to get access to the last element : -1

```
savings = [100, 200, 300, 400, 500]
print(savings[0])
>>> 100
print(savings[-1])
>>> 500
```

- List slicing : [start: end] => include start and exclude end

```
print(savings[1:3])
>>> [200, 300]
print(savings[:3])
>>> [100, 200, 300]
print(savings[1:])
>>> [200,300,400,500]
```

List manipulation?

- Manipulate one or more values within the list (change, add, remove)

```
savings = [100,200,300,400,500]
savings[0] = 800
print(savings)
>>> [800,200,300,400,500]
savings[1:3] = [8000,1000]
print(savings)
>>> [800,8000,1000,400,500]

bigger_savings = savings + [100]
print(bigger_savings)
>>> [800,8000,1000,400,500,100]

del bigger_savings[0]
print(bigger_savings)
>>> [8000,1000,400,500,100]
```

- 04-variables.py
- 05-list.py
- 06-sublist.py
- 07-list_manipulation.py

LET'S PRACTICE



Function?

- A function is a reusable block of code

```
# Custom function
def add_numbers(a,b):
    print(a+b)
add_numbers(1,3)

# built-in function
min(savings)
max(savings)
# need help on the function
help(min)
```

Method?

- A method (method()) is a function linked (by a .) to an object

```
savings = [100, 200, 300, 400, 500]
# Add 100 to the list of savings
savings.append(100) # .append() is a method
print(savings)
>>> [100, 200, 300, 400, 500, 100]
# Count the number of times 100 appears
savings.count(100) #.count is also a method
>>> 2

name = "liz"
name.capitalize()
>>> 'Liz'
name.replace("z","sa")
>>> 'lisa'   # name itself is unchanged (immutable)
```

- 08-function.py
- 09-method.py

LET'S PRACTICE



What's a python package?

- Package is a directory of Python scripts
- Each script of a package is called a module
- Modules specify functions, methods and new Python types aimed at solving particular problems
- Packages for data science:
  - NumPy to efficiently work with arrays
  - Pandas for data analysis
  - Matplotlib for data visualization
  - scikit-learn for machine learning
- Not all these packages are available in Python by default

Install and use a python package?

- Install a package
- Using a command in the terminal : pip (sometimes uv)

```
pip install numpy
```

- Use a package
- In a script, at the top of it, you have to import the package in order to use it

```
import numpy
import numpy as np # better way to import package
# Compute mean
students_mean = np.mean(students)
# Compute mean without as np
students_mean = numpy.mean(students)
```

- Use a module from a package

```
from numpy import random
random.randint(1,10)
```

NumPy?

- Really important for data science (such as pandas as well)
- Numpy : Numerical Python
- List of numbers called numpy arrays (better for data manipulation than basic python list)

```
import numpy as np
height     = [1.85, 1.79, 1.67]
np_height  = np.array(height)
weight     = [65.4, 59.2, 63.6]
np_weight  = np.array(weight)
bmi        = np_weight/np_height**2
print(bmi)
>>> [19.10883857 18.47632721 22.80469002]

weight/height**2 # using basic python list
>>> TypeError: unsupported operand type(s) for ** or pow():
'list' and 'int'
```

NumPy?

- Filter an array ?

```
print(bmi)
>>> [19.10883857 18.47632721 22.80469002]
print(bmi>20)# Which BMI values are above 20?
>>> [False False True]

upper_20 = bmi>20 # Give me all the BMI values above 20
print(bmi[upper_20])
>>> [22.80469002]

print(bmi[bmi>20])
>>> [22.80469002]
```

- 10-packages.py
- 11-numpy.py

LET'S PRACTICE



2D-NumPy?

- A 2D NumPy array means that an array can have multiple rows and columns

```
np_height  = np.array([1.85, 1.79, 1.67])
np_weight  = np.array([65.4, 59.2, 63.6])
type(np_height)
>>> numpy.ndarray # same type for np_weight

np_2d = np.array([[1.85, 1.79, 1.67],[65.4, 59.2, 63.6]])
np_2d.shape
>>>(2,3) # 2 rows, 3 columns
```

2D-NumPy?

- Selecting elements within np_2d

```
np_2d = np.array([[1.85, 1.79, 1.67],
                  [65.4, 59.2, 63.6]])
np_2d[0]
>>> array([1.85, 1.79, 1.67])

np_2d[:, 1:3] # select all rows and columns 1 and 2
>>> array([[ 1.79, 1.67], [59.2 , 63.6 ]])
```

Basic NumPy stats?

```
import numpy as np
np_2d = np.array([[1.85, 1.79, 1.67],
                  [65.4, 59.2, 63.6]])
np.mean(np_2d[:, 0]) # mean of the first column
>>> 33.625

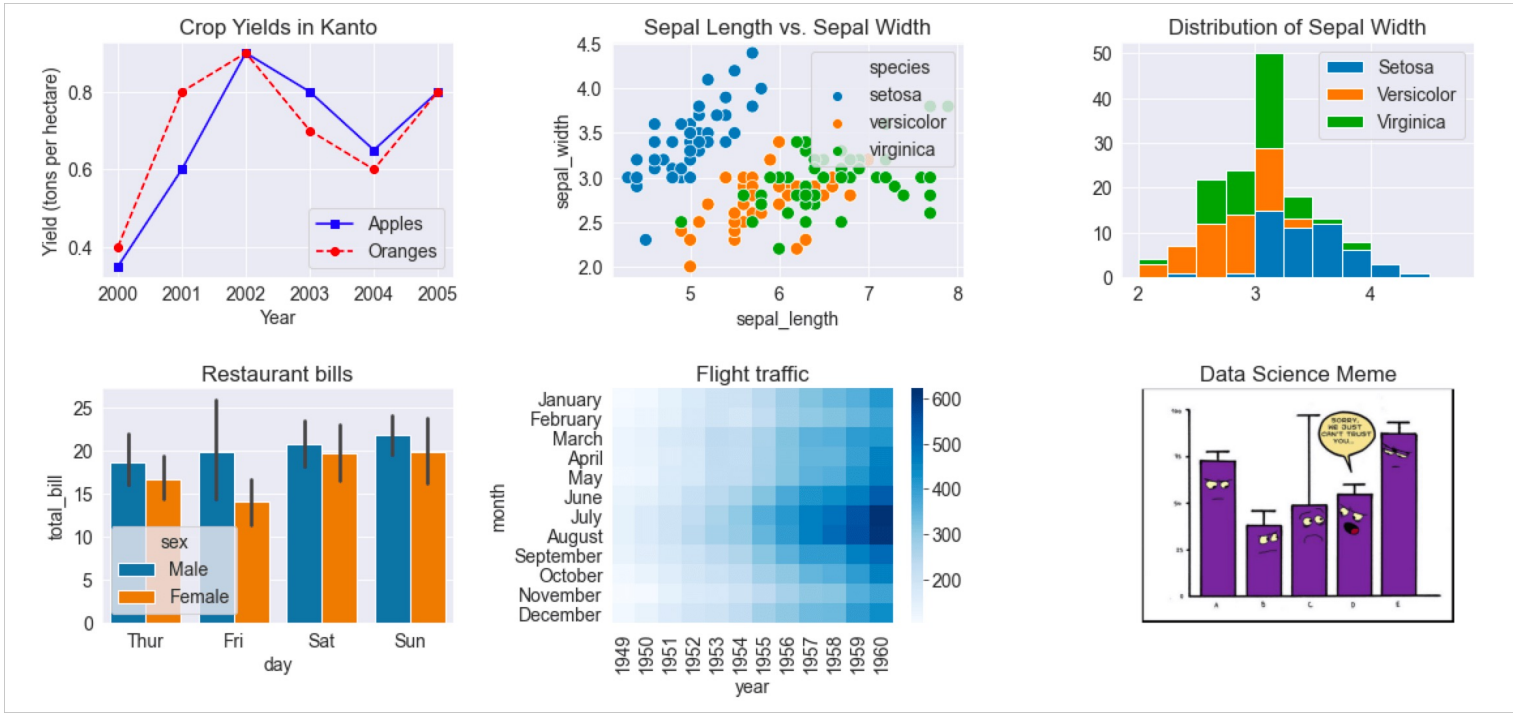
np.std(np_2d[:, 0]) # std of the first column
>>> 31.775

# there's a ton of stats you can do with NumPy … Let's practice!
```

- 12-2D_numpy.py
- 13-numpy_stats.py

LET'S PRACTICE





Source: Northwestern (https://nustat.github.io/DataScience_Intro_python/Data%20visualization.html)

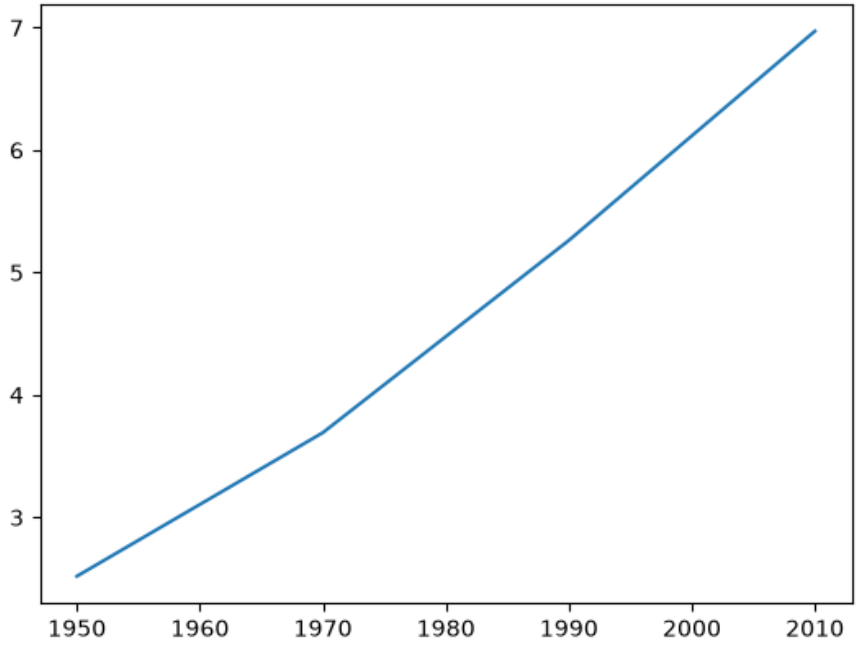
DATA VISUALIZATION



Figures – Matplotlib package!

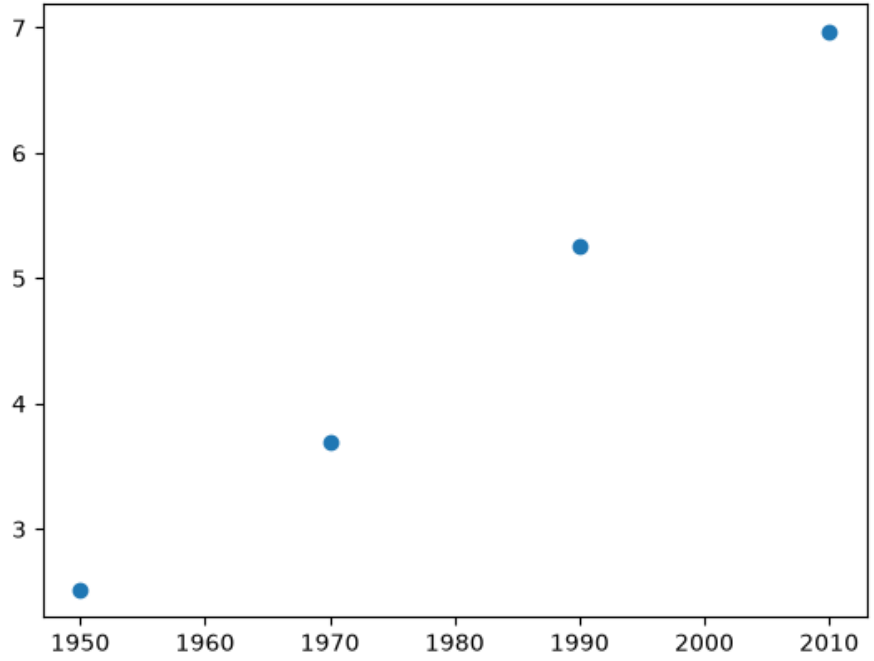
pip install matplotlib (in the terminal)

```
import matplotlib.pyplot as plt
year = [1950, 1970, 1990, 2010]
pop = [2.519, 3.692, 5.263, 6.972]
plt.plot(year, pop)
plt.show()
```



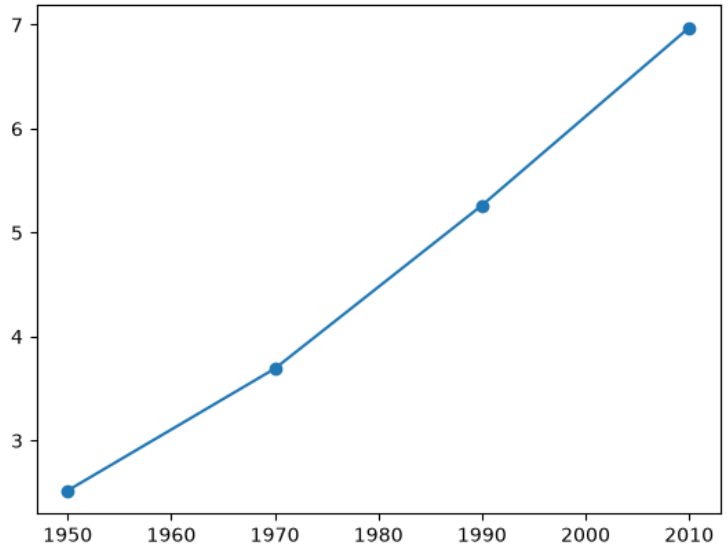
Figures

```
import matplotlib.pyplot as plt
year = [1950, 1970, 1990, 2010]
pop = [2.519, 3.692, 5.263, 6.972]
plt.scatter(year,pop)
plt.show()
```

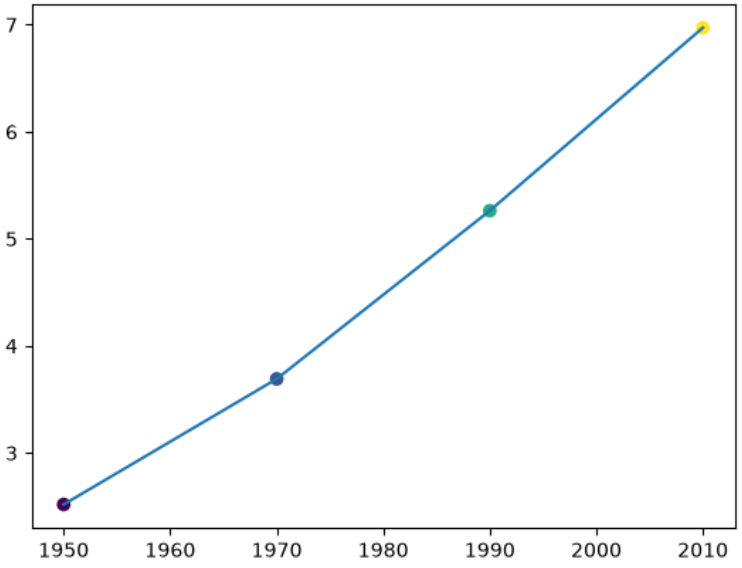


Figures

```
import matplotlib.pyplot as plt
year = [1950, 1970, 1990, 2010]
pop = [2.519, 3.692, 5.263, 6.972]
plt.plot(year, pop)
plt.scatter(year,pop)
plt.show()
```

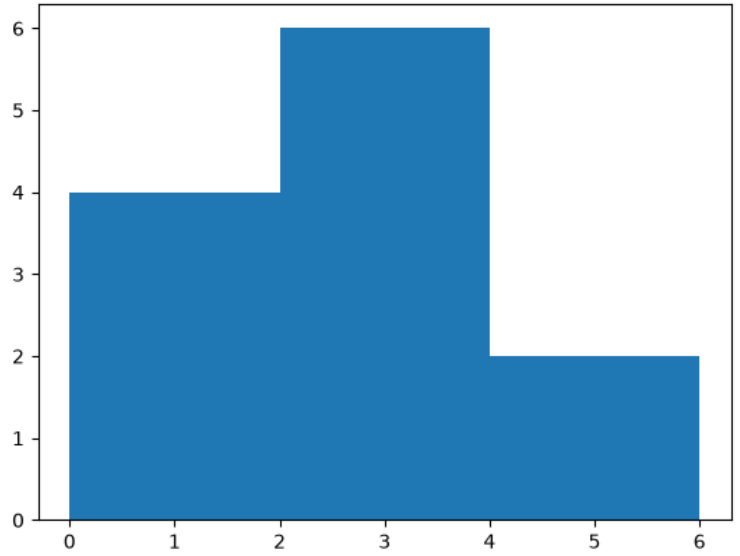



```
import matplotlib.pyplot as plt
year = [1950, 1970, 1990, 2010]
pop = [2.519, 3.692, 5.263, 6.972]
plt.plot(year, pop)
plt.scatter(year,pop,c=pop)
plt.show()
```

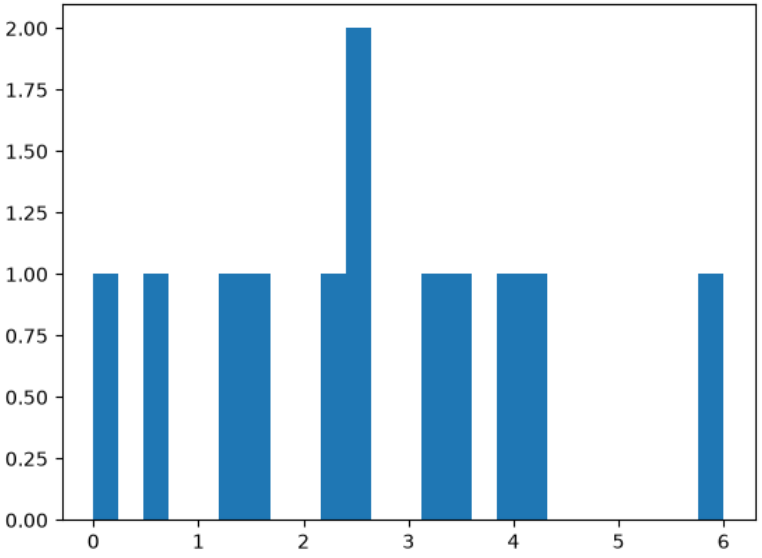


Figures - Histograms

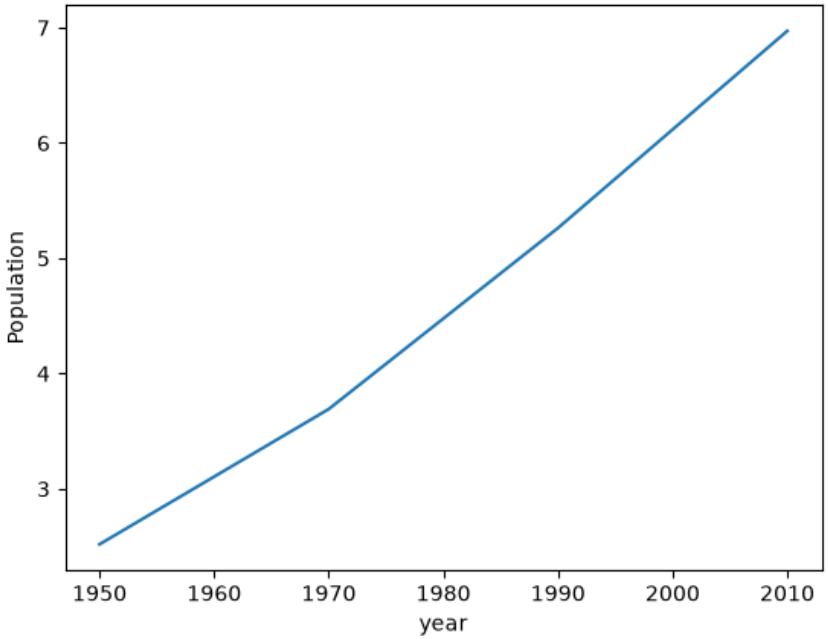
```
import matplotlib.pyplot as plt
values =
[0,0.6,1.4,1.6,2.2,2.5,2.6,3.2,3.5
,3.9,4.2,6]
plt.hist(values, bins=3)
plt.show()
```




```
import matplotlib.pyplot as plt
values =
[0,0.6,1.4,1.6,2.2,2.5,2.6,3.2,3.5
,3.9,4.2,6]
plt.hist(values, bins=25)
plt.show()
# Make really sense?
```

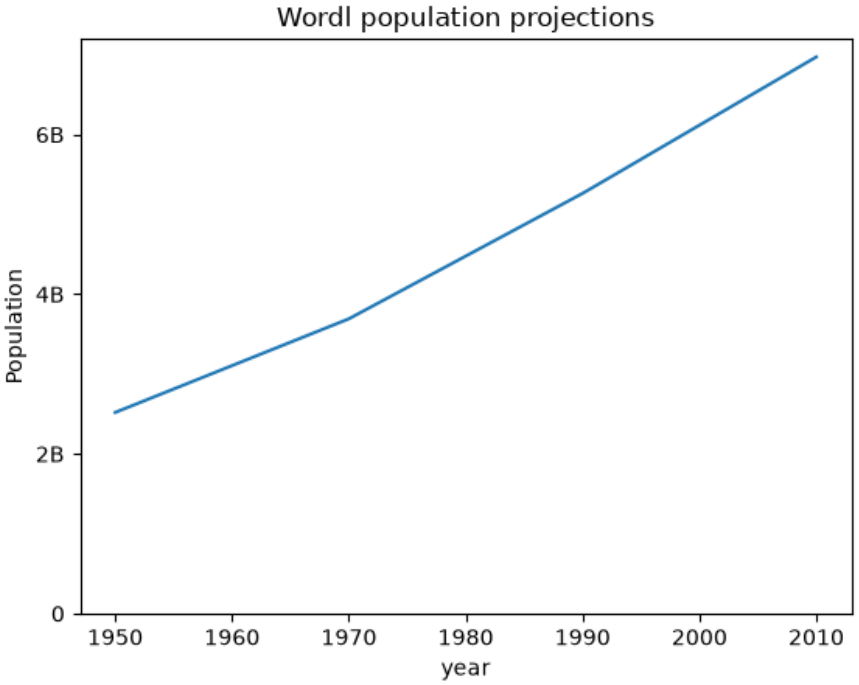


Figures – Customisation




```
import matplotlib.pyplot as plt
year = [1950, 1970, 1990, 2010]
pop = [2.519, 3.692, 5.263, 6.972]
plt.plot(year, pop)
plt.xlabel("year")
plt.ylabel("Population")
plt.show()
```

Figures – Customisation




```
import matplotlib.pyplot as plt
year = [1950, 1970, 1990, 2010]
pop = [2.519, 3.692, 5.263, 6.972]
plt.plot(year, pop)
plt.xlabel("year")
plt.ylabel("Population")
plt.title("World population projections")
plt.yticks([0,2,4,6],["0","2B","4B","6B"]
)
plt.show()
```

There are tons of customization options ! Your job is to find the proper visualization to answer the question/problem

- 14-basic_plots.py
- 15-histogram.py
- 16-customization.py

LET'S PRACTICE



Data manipulation using dictionary!

```
# list of numbers
pop        = [30.55, 2.77, 39.21]
countries  = ["afghanistan", "albania", "algeria"]

# list of countries
ind_alb    = countries.index("albania")
print(ind_alb)
print(pop[ind_alb])
>>> 1
>>> 2.77 # Not really handy
```

Data manipulation using dictionary!

```
# lists
pop        = [30.55, 2.77, 39.21]
countries  = ["afghanistan", "albania", "algeria"]

world = {"afghanistan" : 30.55,
       "albania" :2.77 ,
       "algeria":39.21} # key and a value => dictionary

world["albania"]
>>> 2.77
```

Data manipulation using dictionary!

```
    # add value
    world = {"afghanistan" : 30.55,
            "albania" :2.77 ,
            "algeria":39.21}

    world['sealand'] = 0.000027
    print(world)
    >>> {'afghanistan': 30.55, 'albania': 2.77, 'algeria': 39.21,
'sealand': 2.7e-05}

    # Check a key within the dict
    "sealand" in world
    >>> True
```

Data manipulation using dictionary!

```
    # change a value
    world['sealand'] = 0.000028
    print(world)
    >>> {'afghanistan': 30.55, 'albania': 2.77, 'algeria': 39.21,
'sealand': 2.8e-05}

    # delete an entry
    del world['sealand']
    print(world)
    >>> {'afghanistan': 30.55, 'albania': 2.77, 'algeria': 39.21}
```

Pandas DataFrame?

Country	Capital	Area	Population
Brazil	Brasilia	8.516	200.4
Russia	Moscow	17.10	143.5
India	New Delhi	3.286	1252
China	Beijing	9.597	1357
South Africa	Pretoria	1.221	52.98

- Tabular DataFrame => Rows are observations, columns are variables
- Mix different variable types (str, float etc)
- Built on Numpy (remember 2D-Numpy)
- Great for data manipulation
- Can be created from CSV, dictionary, etc.

Pandas DataFrame?

```
# DataFrame from csv
import pandas as pd
brics = pd.read_csv("/path/brics.csv")

# DataFrame from dict
data = {
    "country": ["Brazil", "Russia", "India",
                "China", "South Africa"],
    "capital": ["Brasilia", "Moscow", "New Delhi",
                "Beijing", "Pretoria"],
    "area": [8.516, 17.10, 3.286, 9.597, 1.221],
    "population": [200.4, 143.5, 1252, 1357, 52.98]
}
brics = pd.DataFrame(data)
```

Access data with Pandas

- 2 methods:
  Square brackets
  Advanced methods: loc & iloc

```
brics["country"]
>>> 0 Brazil 1 Russia 2 India 3 China 4 South Africa

type(brics["country"])
>>> pandas.Series

type(brics)
>>> pandas.DataFrame
```

Access data with Pandas

```
    country        capital        area       population
0   Brazil         Brasilia       8.516      200.40
1   Russia         Moscow         17.100     143.50
2   India          New Delhi      3.286      1252.00
3   China          Beijing        9.597      1357.00
4   South Africa   Pretoria       1.221      52.98
```

```
brics[["country","capital"]]

>>>
   country      capital
0 Brazil       Brasilia
1 Russia       Moscow
2 India        New Delhi
3 China        Beijing
4 South Africa Pretoria
```

Access data with Pandas

```
    country        capital        area       population
0   Brazil         Brasilia       8.516      200.40
1   Russia         Moscow         17.100     143.50
2   India          New Delhi      3.286      1252.00
3   China          Beijing        9.597      1357.00
4   South Africa   Pretoria       1.221      52.98
```

```
brics[1:4]
>>>
    country capital    area population
1   Russia  Moscow     17.100 143.5
2   India   New Delhi  3.286 1252.0
3   China   Beijing    9.597 1357.0
```

Access data with Pandas

```
    country        capital        area       population
0   Brazil         Brasilia       8.516      200.40
1   Russia         Moscow         17.100     143.50
2   India          New Delhi      3.286      1252.00
3   China          Beijing        9.597      1357.00
4   South Africa   Pretoria       1.221      52.98
```

```
    brics.index = ["BR", "RU", "IN", "CH", "SA"]
    brics
    >>>
```

```
    country        capital        area       population
BR   Brazil        Brasilia       8.516      200.40
RU   Russia        Moscow         17.100  143.50
IN   India         New Delhi      3.286      1252.00
CH   China         Beijing        9.597      1357.00
SA   South Africa  Pretoria       1.221      52.98
```

Access data with Pandas

	country	capital	area	population
BR	Brazil	Brasilia	8.516	200.40
RU	Russia	Moscow	17.100	143.50
IN	India	New Delhi	3.286	1252.00
CH	China	Beijing	9.597	1357.00
SA	South Africa	Pretoria	1.221	52.98

```
brics.loc["RU"]
>>> country Russia
capital Moscow
area 17.1
population 143.5
Name: RU,
dtype: object # pandas.Series

brics.loc[["RU"]] #pandas.DataFrame
>>> country capital area population
RU Russia Moscow 17.1  143.5
```

Access data with Pandas

	country	capital	area	population
BR	Brazil	Brasilia	8.516	200.40
RU	Russia	Moscow	17.100	143.50
IN	India	New Delhi	3.286	1252.00
CH	China	Beijing	9.597	1357.00
SA	South Africa	Pretoria	1.221	52.98

```
brics.loc[["RU", "BR","SA"]]
>>>
   country     capital        area    population
RU Russia      Moscow  17.100 143.50
BR Brazil      Brasilia       8.516   200.40
SA South Africa Pretoria      1.221   52.98

brics.loc[["BR", "RU"],["country","capital"]]
>>> country capital
    BR Brazil Brasilia
    RU Russia Moscow
```

Access data with Pandas

	country	capital	area	population
BR	Brazil	Brasilia	8.516	200.40
RU	Russia	Moscow	17.100	143.50
IN	India	New Delhi	3.286	1252.00
CH	China	Beijing	9.597	1357.00
SA	South Africa	Pretoria	1.221	52.98

```
 brics.iloc[0,0]
>>> Brazil

brics.iloc[:,0]
>>>
BR Brazil
RU Russia
IN India
CH China
SA South Africa
Name: country, dtype: str
```

- 17-dictionary.py
- 18-pandas.py

LET'S PRACTICE



Comparison operators

```
# Boolean response
2<3
>>> True

2 == 3
>>> False

2 <= 3
>>> True

x = 2
y = 3
y > x
>>> True
```

```
# Boolean response
"carl" < "chris"
>>> True

3 < "chris"
>>> TypeError!

3 < 4.1
>>> True

bmi = np.array([21,20,35,19.5])
bmi > 23
>>> array([False, False, True,
False])
```

Comparison operators

Comparator	Meaning
<	Strictly less than
<=	Less than or equal
>	Strictly greater than
>=	Greater than or equal
==	Equal
!=	Not equal

Boolean operators

```
# AND
True and True
>>> True

False and True
>>> False

# Find a value between two limits ? (example)
x = 12
x > 5 and x < 15
>>> True
```

Boolean operators

```
# OR
True or True
>>> True

False or True
>>> True

# Find a value outside two limits ? (example)
x = 20
x < 5 or x > 15
>>> True
```

Boolean operators

```
# not
not True
>>> False

not False
>>> True
```

Boolean operators

```
# filtering data
bmi = np.array([21,20,35,19.5])
bmi > 23
>>> array([False False True False])

bmi <30
>>> array([True True False True])

bmi > 20 and bmi < 30 # Warning when combining operators
>>> ValueError: The truth value of an array with more than one
element is ambiguous. Use a.any() or a.all()
# => LIMITS OF CLASSIC PYTHON

(bmi > 20) & (bmi < 30) # NumPy
>>> array([True False False False])
```

Boolean operators

```
# NumPy
 (bmi > 20) & (bmi < 30)
>>> array([True False False False])

# NumPy
np.logical_and(bmi>20 , bmi<30)
 >>> array([True False False False])

# & works on bits, logical_and works on truth values.
On comparisons (already boolean), both give the same result,
so use &.
```

Boolean operators

Python	Pandas / NumPy
and	& or np.logical_and
or	| or np.logical_or
not	~ or np.logical_not

If, elif, else operators

```
if condition :
    expression

# example
if z == 3:
    print("victory")

# when z = 3
>>> victory
# when z = 4
>>> …
```

```
# add a condition : else
if z == 3:
    print("victory")
 # all other cases
else:
    print("defeat")

# when z = 3
>>> victory
# when z = 4
>>> defeat
# when z = 5
>>> defeat
```

If, elif, else operators

```
# add a specific condition :  elif

if z == 3:
    print("victory")
# evaluated only if the previous condition is False
elif z == 4:
    print("draw")
# all other cases
else:
    print("defeat")

# when z = 3
>>> victory
# when z = 4
>>> draw
# when z = 5
>>> defeat
```

- 19-comparison_operators.py
- 20-boolean_operators.py
- 21-condition_operators.py

LET'S PRACTICE



Filtering a dataframe

```
# DataFrame from csv
import pandas as pd
brics = pd.read_csv("/path/brics.csv")
```

```
    country        capital        area       population
0   Brazil         Brasilia       8.516      200.40
1   Russia         Moscow         17.100     143.50
2   India          New Delhi      3.286      1252.00
3   China          Beijing        9.597      1357.00
4   South Africa   Pretoria       1.221      52.98
```

```
# Select countries with area over 8 millions km2 ?
# Step 1 : get the column
# Step 2 : do comparison on the column
# Step 3 : use result to select the countries
```

Filtering a dataframe

```
    country        capital        area       population
0   Brazil         Brasilia       8.516      200.40
1   Russia         Moscow         17.100     143.50
2   India          New Delhi      3.286      1252.00
3   China          Beijing        9.597      1357.00
4   South Africa   Pretoria       1.221      52.98
```

```
import pandas as pd
brics = pd.read_csv("/path/brics.csv")
# Step 1: get the column
brics["area"]                    # a pandas Series
# Step 2: do comparison on the column
is_huge = brics["area"] > 8   # boolean Series
# Step 3: use the result to select the countries
brics[is_huge]
>>>
   country  capital  area  population
 0  Brazil  Brasilia  8.516  200.40
 1  Russia  Moscow   17.100  143.50
 3  China   Beijing   9.597 1357.00
```

Filtering a DataFrame

```
    country        capital        area       population
0   Brazil         Brasilia       8.516      200.40
1   Russia         Moscow         17.100     143.50
2   India          New Delhi      3.286      1252.00
3   China          Beijing        9.597      1357.00
4   South Africa   Pretoria       1.221      52.98
```

```
# faster way
brics[brics["area"] > 8]
>>>
   country  capital  area  population
 0  Brazil  Brasilia  8.516  200.40
 1  Russia  Moscow   17.100  143.50
 3  China   Beijing   9.597 1357.00
```

Filtering a DataFrame

```
# other ways
import numpy as np
brics[np.logical_and(brics["area"] > 8, brics["area"] < 15)]
>>>
   country  capital  area  population
 0  Brazil  Brasilia  8.516  200.40
 3  China   Beijing   9.597 1357.00

brics[(brics["area"] > 8) & (brics["area"] < 15)]  # operator &
>>>
 country  capital  area  population
 0  Brazil  Brasilia  8.516  200.40
 3  China   Beijing   9.597 1357.00
```

- 22-filtering_dataframe.py

LET'S PRACTICE



While loop

```
while condition: # as long as the condition is True
    expression

error = 50.0
while error > 1:
    error = error /4
    print(error)

>>> 12.5
>>> 3.125
>>> 0.78125 # the loop stops: error < 1
```

While loop

```
# warning infinite loop : AVOID !!!!
error = 50.0
while error > 1: # always true
        print(error)

>>> 50.0
>>> 50.0
>>> …
```

For loop

```
for var in sequence:
    expression

bmi = [19.7,40,28,23]
# print each value instead of the whole list
for value in bmi:
    print(value)

>>>
19.7
40
28
23
```

For loop

```
bmi = [19.7,40,28,23]
for index,value in enumerate(bmi):
    print("index " + str(index) + " : " + str(value))

>>>
index 0 : 19.7
index 1 : 40
index 2 : 28
index 3 : 23

for index,value in bmi: # WARNING
    print("index " + str(index) + " : " + str(value))
>>> TypeError: cannot unpack non-iterable float object
```

For loop

```
for c in "family":
    print(c.capitalize())
```

```
F
A
M
I
L
Y
```

- 23-while_loop.py
- 24-for_loop.py

LET'S PRACTICE



For loop on a data structure

```
# dict
world = {"afghanistan" : 30.55,
        "albania" :2.77 ,
        "algeria":39.21}

# select each key + value of the dict
for key, value in world.items():
    print(key + " : " + str(value))

>>>
    afghanistan : 30.55
    albania : 2.77
    algeria : 39.21
```

For loop on a data structure

```
# dict
world = {"afghanistan": 30.55,
         "albania": 2.77,
         "algeria": 39.21}
bmi = [21, 20, 35]

# items() on a dict: gives (key, value)
list(world.items())
[('afghanistan', 30.55), ('albania', 2.77), ('algeria', 39.21)]

# enumerate on a list: gives (index, value)
list(enumerate(bmi))
[(0, 21), (1, 20), (2, 35)]

# enumerate on a dict: gives (index, KEY), not the value
list(enumerate(world))
[(0, 'afghanistan'), (1, 'albania'), (2, 'algeria')]
```

For loop on a data structure

```
import numpy as np

np_height = np.array([1.73, 1.68, 1.71, 1.89, 1.79])
np_weight = np.array([65.4, 59.2, 63.6, 88.4, 68.7])

bmi = np_weight / np_height ** 2

for val in bmi:
    print(f"{val:.3f}")

>>>
21.852
20.975
21.750
24.747
21.441
```

For loop on a data structure

```
import numpy as np

np_height = np.array([1.73, 1.68, 1.71, 1.89, 1.79])
np_weight = np.array([65.4, 59.2, 63.6, 88.4, 68.7])

meas = np.array([np_weight,np_height])

for val in meas:
    print(val)

>>>
[65.4 59.2 63.6 88.4 68.7]
[1.73 1.68 1.71 1.89 1.79]
```

For loop on a data structure

```
import numpy as np

np_height = np.array([1.73, 1.68, 1.71, 1.89, 1.79])
np_weight = np.array([65.4, 59.2, 63.6, 88.4, 68.7])

meas = np.array([np_weight,np_height])

# iterate over each element
for val in np.nditer(meas):
    print(val)

>>>
65.4
59.2
…
```

Loop on a dataframe

```
    country        capital        area       population
BR   Brazil        Brasilia       8.516      200.40
RU   Russia        Moscow         17.100     143.50
IN   India         New Delhi      3.286      1252.00
CH   China         Beijing        9.597      1357.00
SA   South Africa  Pretoria       1.221      52.98
```

```
import pandas as pd
# import csv and use column 0 as index
brics = pd.read_csv("/path/brics.csv", index_col=0)
for val in brics: # equivalent as brics.columns (works as a
dict)
    print(val)
>>>
country
capital
area
population
```

For loop on a dataframe

```
import pandas as pd

# import csv and use column 0 as index
brics = pd.read_csv("/path/brics.csv", index_col=0)

for lab, row in brics.iterrows():
    print(lab)   # label
    print(row)   # row (Series)
>>>
BR
country        Brazil
capital      Brasilia
area            8.516
population      200.4
Name: BR, dtype: object
# dtype: object because the row mixes str and float

RU …
```

For loop on a dataframe

```
for lab, row in brics.iterrows():
    print(lab + ":" + row["capital"])
```

```
>>>
BR:Brasilia
RU:Moscow
IN:New Delhi
CH:Beijing
SA:Pretoria
```

- 25-loop_datastructure.py
- 26-loop_dataframe.py

LET'S PRACTICE



WRITE YOUR OWN FUNCTION



Write your own function

- A function has a name, parameters and a body. return sends a value back to the caller

```
def add_numbers(a, b):              # a, b are parameters
    return a + b                    # send the result back

result = add_numbers(1, 3)          # 1, 3 are arguments
print(result)
>>> 4

# print() only displays a value, return gives it back
def add_print(a, b):
    print(a + b)

x = add_print(1, 3)                 # displays 4
print(x)
>>> None
```

Parameters and return values

- A function can take several parameters and return several values (as a tuple)

```
def bmi(weight, height):
    """Return the body mass index"""      # docstring: help text
    return weight / height ** 2

round(bmi(65.4, 1.85), 2)
>>> 19.11

def min_max(values):
    return min(values), max(values)       # returns a tuple

low, high = min_max([19.7, 40, 28, 23])   # tuple unpacking
print(low, high)
>>> 19.7 40
```

Counting with a dictionary

- Pattern: loop over the values and update a dictionary (key = value, count = occurrences)

```
languages = ["en", "fr", "en", "es", "en"]
counts = {}

for lang in languages:
    if lang in counts.keys():
        counts[lang] = counts[lang] + 1    # key exists: add 1
    else:
        counts[lang] = 1                   # new key: start at 1

print(counts)
>>> {'en': 3, 'fr': 1, 'es': 1}
```

Function on a DataFrame

- Pass the DataFrame and a column name as parameters, loop over the column, return the result

```
def column_mean(df, col_name):
    """Return the mean of a numeric column"""
    total = 0
    for value in df[col_name]:
        total = total + value
    return total / len(df)

round(column_mean(brics, "area"), 3)
>>> 7.944

import matplotlib.pyplot as plt
plt.bar(brics["country"], brics["area"])    # bar plot
plt.show()
```

- 27-user_defined_function.py
- 28-multi_params.py
- 29-final_script.py

LET'S PRACTICE



END COURSE EVALUATION



End course evaluation

- In PYTHON_101 folder lies the project folder containing the script template with all the instructions
- Rename the template with your name and PYTHON_101,
  ex: VANNSON_PYTHON_101.py
- When done, upload it on campus
- You'll have a week, deadline : 23/10/2026 @23h59



vannson.nicolas@gmail.com